SIP-55 - [MU] E-invoicing Implementation

Reference Tickets

SIP-55: Mauritius - Fiscal CR - E-invoicing Implementation PENDING MARKET

Revisions	Updates	Author	Date
1.0	Initial Draft	@Venkata Madira	21 Oct 2025
1.1	Rogers Capital Updates	@Venkata Madira	31 Oct 2025

Problem Statements

As a Market User,
I want to integrate **e-invoicing capabilities** into the **NP6 system**,
So that **receipts include a digital QR code**,
Enabling **automated electronic submission of invoices** and ensuring **seamless compliance with government regulations**.

Summary

The **Mauritius Revenue Authority (MRA)** has mandated the adoption of **e-invoicing** to enhance tax compliance, streamline business transactions, and improve financial reporting. In response, our organization must modernize its invoicing processes to align with these new regulatory requirements.

Currently, our invoicing system is **manual and paper-based**, leading to frequent errors, delays, and inefficiencies. This outdated approach increases the risk of **non-compliance**, potential **penalties**, and adds significant **administrative overhead**, while also negatively impacting **cash flow** and **reporting accuracy**.

Implementing an **e-invoicing solution** will ensure full compliance with MRA regulations, reduce operational risks, and improve overall efficiency. The transition will automate invoice generation, transmission, and archiving, resulting in:

- **Fewer manual errors**
- **Lower administrative costs**
- **Faster invoice processing and payments**
- **Improved data accuracy and reporting**

This strategic initiative will not only meet regulatory demands but also support better financial decision-making and enhance our organization's agility in managing business transactions.

Business Impact of Not Implementing E-Invoicing

1. **Legal and Compliance Risks**
   - Exposure to **fines, penalties, or sanctions** from the Mauritius Revenue Authority due to non-compliance with mandatory e-invoicing regulations.
2. **Operational Inefficiencies**
   - Continued use of **manual, paper-based invoicing** leads to frequent **errors**, **processing delays**, and **high administrative overhead**.
3. **Financial Losses**
   - Increased risk of **delayed payments**, **billing inaccuracies**, and **additional costs** from manual corrections and follow-ups.
4. **Limited Data and Reporting Capabilities**
   - Inability to generate **accurate, timely financial reports**, which undermines **strategic decision-making** and **cash flow visibility**.
5. **Regulatory Penalties**
   - Specific fines or sanctions may be imposed by the government depending on the **severity and duration of non-compliance**, potentially impacting reputation and financial stability.

Requirements

Business Requirements	Details
BR-1	**Real-Time Validation with InVox Server** The system must validate each transaction instantly with the InVox's tax server before completing the sale, ensuring compliance and authenticity.
BR-2	**Automatic QR Code Generation and Printing** The system must generate a compliant QR code containing transaction details and print it on the customer receipt for verification purposes.
BR-3	**Secure and Encrypted Data Transmission** All communication between the POS system and the government server must be securely encrypted to protect sensitive transaction data and ensure integrity.
BR-4	**Robust Error Handling and User Alerts** The system must provide clear notifications and guidance to the cashier in case of validation failures, network issues, or printer errors, minimizing transaction disruptions.
BR-5	**Refund Processing and Reversal for E-Invoicing with Government Tax Validation**

Hardware Dependencies

- N/A

Out of Scope

- Registration process
- Waste
- Overring
- MOP
- BR-3 - There will be no encryption between NewPOS / InVox, only the signing of the json. Encryption will be in place between InVox and MRA, outside of NewPOS.
- invoiceTypeDesc such as **DRN (Debit Note), PRF (Proforma), or TRN (Training)**
- B2B transactions are out of scope, there are currently no B2B transactions in any MCD restaurants
- Mixed payments such as two or more tenderTypes used in single transaction

Operating Environment

- POS (FC/DT/MCC)
- Kiosk#
- Waystation
- 3PO Non-integrated

Attachments

Please note, the MRA documents below are for information purposes only as these documents are directly from the Mauritius Revenue Authority. NewPOS will not be interacting directly with the MRA. Instead, NewPOS will interact with InVox, therefore for development purposes, the InVox documentation below should be referred to.

InVox Technical Documentation.pdf

InVox DataStructure.cleaned.pdf

MRA TechnicalGuidelines.pdf

MRA UserGuidelines.pdf

MRA GeneralGuidelinesSoftwareDevelopers.pdf

MRA DataStructure.pdf

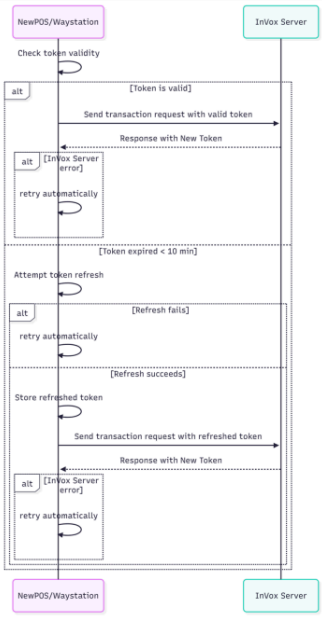
Glossary of Terms

Terms	Explanations
EBS	Electronic Billing System
MRA	Mauritius Revenue Authority
MRAID Portal	MRA e-Invoicing Developer Portal
IRN	Invoice Registration Number

Use Cases

Use Case 1	**Secure and Signed Data Transmission** All communication between the POS system and the InVox server must be signed to protect sensitive transaction data and ensure integrity.
Rational	To ensure secure and compliant fiscal operations, NewPOS must authenticate with the InVox service at the start of each business day. This integration allows the POS system to obtain a valid access token, which is then used to authorize all fiscal transactions (e.g., receipt generation, refunds) throughout the day.
Reference	
Steps to Follow:	**NP6 POS/POS(DLV)/Kiosk communicate with InVox via NP6 Waystation to get Authorized Token:** An authentication token request should be automatically triggered whenever a cashier log-in on Newpos system. This ensures that the system is authorized and ready to securely process transactions through the INVOX e-Invoicing APIs. Before transmitting any transaction data to the **INVOX e-Invoicing System**, a registered **POS** must authenticate itself using the **Authentication API**. **Key Steps:** **Step 1: Business Day Open Trigger** • At the start of the business day, Waystation initiates a system-level authentication request to InVox. **Step 2: Token Request to InVox** • Waystation sends a `POST` request to the InVox token endpoint. • Request includes: ◦ `grant_type` : `client_credentials` ◦ `client_id` , `client_secret` , and `scope` : Provided by RCTS **Step 3: Token Response Handling** • InVox returns a JSON response with: ◦ `access_token` ◦ `token_type` (Bearer) ◦ `expires_in` (validity in seconds) **Step 4: Token Storage** • Waystation securely stores the token in local DB and distributes it to the individual nodes such as POS/VPOS(It may not be required today, but it could be necessary in the future.)/CSO. • The token is reused for all API Requests to InVox during the business day. **Step 5: Token Renewal** • If the token expires during the day, Waystation automatically re-authenticates using the same credentials to obtain a new token and distribute to all connected nodes. **Step 6: Error Handling** • If token request fails: ◦ Log the error

	◦ Display message on POS, it will not perform any further transactions ◦ Retry based on error type (e.g., network issue, invalid credentials)
User Interface	
Technical Interface	The sequential diagram will illustrate the technical flow. Currently, the Mauritius market is classified under fiscal category 5, but with the implementation of this feature, it will be reclassified to category 3.



Token Request Sequential Diagram

Steps to Generate the Authentication JSON:

Endpoint/ URL:

Sample URL: POST https://auth.rcts.example.com/token

Headers	Body Parameters	Sample Response

Content-Type: application/x-www-form-urlencoded	grant_type=client_credentials
	client_id=<provided_by_RCTS>
	client_secret=<provided_by_RCTS>
	scope=<provided_by_RCTS>
		`{`
		
		`"token_type": "Bearer",`
		
		`"expires_in": 3599,`
		
		`"ext_expires_in": 3599,`
		
		`"access_token": "<JWT_ACCESS_TOKEN>"`
		
		`}`
		
		To authenticate subsequent API requests, include the token in the `Authorization` header

Token Refresh Logic: Waystation/POS must check the **token's expiry time**. If the remaining validity is **10 minutes or less**, then the Waystation sends a request to the InVox Server to get new token and use it for future API requests.

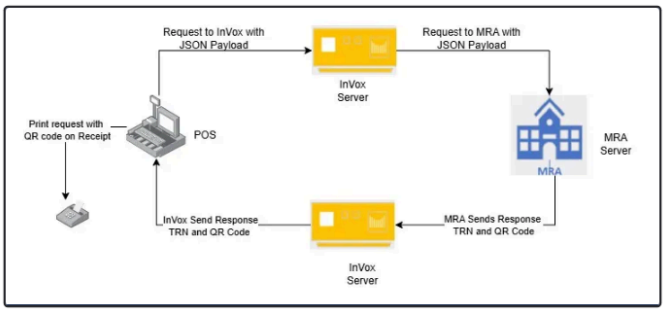
Token Management Overview

- The **Token Manager** refreshes the token every `X` **minutes**.
- On successful refresh:
  - Saves the new token and timestamp in **RavenDB**.

	◦ Broadcasts the token to all connected devices. **Waystation & POS Behavior** • On receiving a new token: ◦ Updates local **RavenDB** with the token and timestamp. ◦ Uses the token for **SALE/Refund API requests**. • If no token is received: ◦ Calls `requestToken()` from the Token Manager. ◦ If the current token is valid and updated within `y` **minutes**, it is returned. ◦ Otherwise, a new token is generated, stored, and returned. **Best practice Token Refresh Logic in Waystation:** • POS checks the token's expiry time before each transaction. • If the remaining validity is **10 minutes or less**, **Waystation** sends a request to the **InVox Server** to fetch a new token. • The new token is distributed to all nodes POS/VPOS/CSO and then is used for all future API requests. **Note:** x and y are values representing hours and minutes. For example x = 24*60 -10 (minutes) and y = 10 (minutes). X is the token validity which has been recommended by InVox to refresh every 24 hours. This is non-configurable. Y is the minutes that should be configurable in order to request a new token, therefore a new parameter should be created for this. The recommended value is "10". Default value should also be "10".
Samples:	
Configuration Attributes	• **URL Path - New parameter to know** POST URL Ex. https://auth.rcts.example.com/token • grant_type=client_credentials • client_id=<provided_by_RCTS> • client_secret=<provided_by_RCTS> • scope=<provided_by_RCTS>

Use Case 2	NP6 Kiosk to get the Authentication Token
Assumptions	Kiosk is working as expected without any network outages.
Steps to Follow:	**Token Request and Validation Workflow** • **On Business Day open:** Immediately after a Business day open in—whether through waystation or Backoffice —a **new authentication token request** must be initiated, as outlined in **Case 1**. • **Before Transaction Processing:** Prior to executing any payment transaction, **CSO** must verify the **token's expiry time**. ◦ If the token has **10 minutes or less remaining**, the Waystations must initiate a new token request. ◦ If the token is valid for **more than 10 minutes**, the transaction request can be proceed.
User Interface	
Technical Interface	**Please refer to Use case 1**
Configuration Attributes	**Please refer to Use case 1**

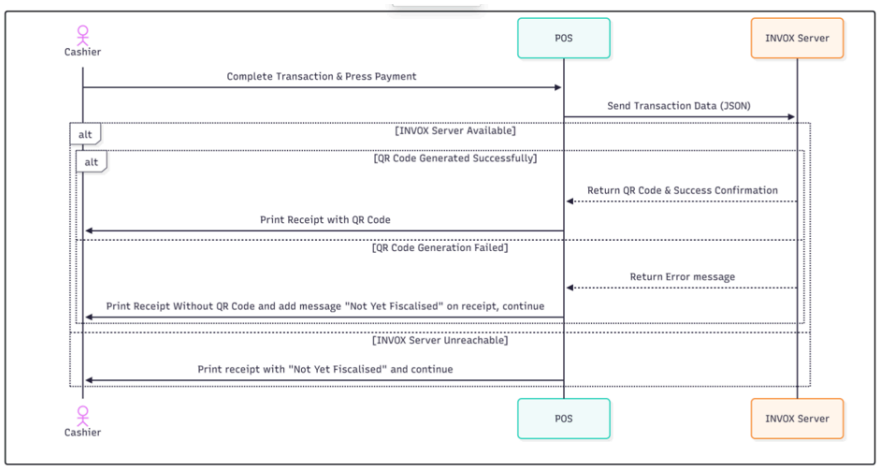
Use Case 3	**Newpos (Every POD Device) Real-Time Validation with InVox Server** The NewPOS must send each transaction to the InVox server after completing the payment done, ensuring compliance and authenticity.
Assumptions	• The POS system is compatible with the government's e-invoicing and QR code generation specifications.

	• The government tax server is operational, reliable, and available during business hours. • The POS system has a stable internet connection to communicate with the InVox server in real-time. • Proper security protocols are in place to protect transaction data during transmission. • All relevant staff are trained on the new process of transaction authentication and QR code printing. • The QR code printed on the receipt accurately encodes the transaction verification details as per government requirements. • The necessary hardware (such as printers capable of printing QR codes) and software updates are installed. • The government's e-invoicing system remains compliant with any future updates or changes to regulations.
Steps to Follow:	**Precondition for Transaction Requests to InVox Server** • A **valid business day** must be opened. • A **cashier must be successfully logged in**. • A **valid authentication token** must be available before initiating any transaction request to the **InVox server**. Additionally, the system must verify that the token has **more than 10 minutes of validity remaining** before processing any transaction. If not, a **token refresh** must be triggered. **Transaction and MRA Submission Workflow** 1. **Add Items** ◦ Select and add items to the cart in the **CSO** system. or ◦ Add items to sales panel in the POS system. 2. **Choose Order Type** ◦ Select either **Eat-in** or **Take-out**. 3. **Select Payment Method** ◦ On the **Tender screen**, choose the appropriate **payment type**. 4. **Prepare JSON Data Template** ◦ Generate the required **transaction data** in JSON format. Please refer to 'JSON Template to be prepared from NewPOS' section: SIP-55 - [MU] E-invoicing Implementation | JSON Template to be prepared from NewPOS 5. **Send Request to InVox Server** ◦ Transmit the JSON data to the **InVox e-Invoicing server**. 6. **Await Response** ◦ Wait for the server's response to confirm transaction verification and QR code generation.
User Interface	
Technical Interface	**Process Flow:**  1. **POS** validates the token expiry time. Once it is valid then go to next step. Otherwise send request to Waystation to get latest token. 2. **POS System** prepares the request with transaction as JSON payload. 3. **POS** creates the API request to the **InVox Server with RAW JSON (locally signed)**. Please refer to SIP-55 - [MU] E-invoicing Implementation | JSON Template to be prepared from NewPOS 4. **InVox Server** relays the request to the **MRA Server**. 5. **MRA Server** processes the request and sends a response back to the **InVox Server**.

6. **InVox Server** returns the response to the **POS**.
7. POS will generate the receipt with QR code on it.
   a. If the InVox server has not responded within time period then POS will generate receipt with message “Not Yet Fiscalised“ and POS will continue to take orders. The text should be displayed in the same position where the QR Code image was intended to be displayed.
   b. Save the JSON data in unprocessed folder, used for retry mechanism.

Transaction submission via InVox Transmission API

Sequential diagram will explain the steps to follow.



Steps to Generate a SALE Authorization request from POS to InVox :

Endpoint/ URL:

Sample URL: POST https://auth.rcts.example.com/sale

Headers	Body Parameters	Sample Response
Content-Type -application/json	Prepared JSON as per template JSON Template to be prepared from NewPOS	Sample JSON response only

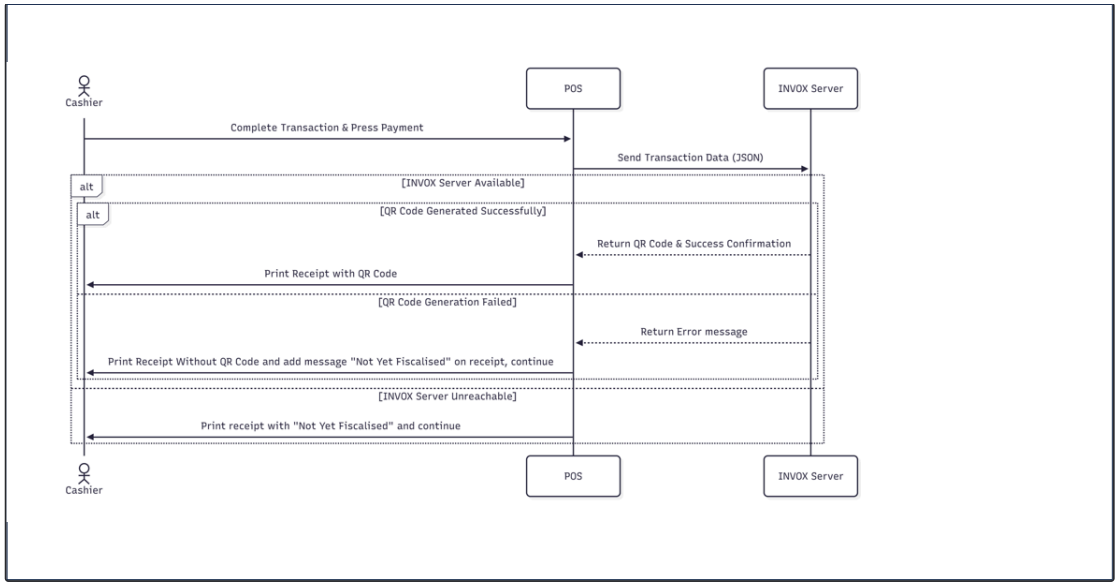

```
{
    "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/1803b764-fc06-4f4d-b5e0-f311d9826663/Prototype/api/RogersCapital/MraInvoice/v2.0/$metadata#companies(019d7222-52e9-ed11-884d-6045bdc8c698)/mrainvoice/$entity",
    "@odata.etag": "W/\"JzE5OzM2MzQ1NDc0MzM4NjUyMzQ3NTgxOzAwOyc=\"",
    "no": "INVOX0140",
    "buyerBRN": "C07027054",
    "buyerBusinessAddress": "Rogers House Port-Louis",
    "buyerName": "Rogers Capital Technology Services Ltd",
    "buyerTAN": "20216629",
    "buyerTypeMra": "VATR",
    "cashierID": "4180",
    "currency": "MUR",
  ..............................
  ..............................
  ..............................
  ..............................
    "qrcodebase64": "iVBORw0KGgoAAAANSUhEUgAAAV4AAAFeAQAAAAD1UEq3AAACCElEQVR4Xu2YMXLDMAwEkUnhUk
/QU/w06216ip6gUkUmDO5AKoidybiIyeaukCBiUd2ABGXleX3Y/cofEpwlOEtwluAswVmCswRnVXi30Owrb8Vuqwf
l03NrTdwE94cvfK3zfnEYzHTYWVWK4CHwB2z6kWLg5cYqwePgSDXRU8Gj4XK8c6F4wL1O8ECYL6Sc8dQ2pZOoNZrg
```

```
vnDto3nHAQSmBTUhuD/8rcO4xZnB02BOCc56ORwO0i9scYvVUc2u2xSbnuARMMZmD6KbEMS1ZuY2eN+DgnvAYNBNPHdcMK7QQcPAQEhwX7jEZZ",
    "mrainvoicelines": [
        {
            "@odata.etag": "W/\"JzIwOzE3NTQ4NzM4NTk4MjEyMTEzMzU1MTswMDsn\"",
            "documentNo": "INVOX0140",
            "lineNo": 10000,
            "amtWoVatCur": "17000",
            "amtWoVatMur": "17000",
            "discount": "15"
          ...............................
          ...............................
          ...............................
        }
      ...............................
      ...............................
      ...............................
    ]
}
```


Note: InVox infra setup was not yet done so the URL and its associated properties will change accordingly.

Configuration Attributes	• **URL Path** - **New parameter to know** POST URL Ex. https://auth.rcts.example.com/sale • grant_type=client_credentials • client_id=<provided_by_RCTS> • client_secret=<provided_by_RCTS> • scope=<provided_by_RCTS> **Note**: Complete configuration Setup will be found at the Release notes only.

Use Case 3	**Automatic QR Code Generation and Printing** The system must generate a compliant QR code containing transaction details and print it on the customer receipt for verification purposes.
Assumptions	• The POS system is configured to connect with the Mauritius government tax department's server. • The POS system has the capability to generate and print QR codes. • Network connectivity between POS and the government server to be established and tested.
Steps to Follow:	**Steps to follow:** 1. **Complete Token Authentication** After successfully obtaining a valid authentication token as described in **Use Case 1**, proceed to the next step. 2. **Submit Sales Transaction** Complete the sales transaction process as outlined in **Use Case 2**, including preparing and sending the transaction JSON data to the InVox server. 3. **Print QR Code on Receipt** Upon receiving a successful response from the InVox server, extract the QR code from the response and print it on the customer receipt footer section.
User Interface	
Technical Interface	**Sequential Diagram to follow:**



POS Transaction and Receipt Generation Flow:

1. POS System Prepares the Request

- The **POS system** gathers transaction details (e.g., items purchased, prices, payment method).
- It formats this data into a **JSON payload**, which is a structured format suitable for transmission.
- This payload includes metadata like InvoiceID, timestamp, and possibly transaction info. **Please refer to** SIP-55 - [MU] E-invoicing Implementation | JSON Template to be prepared from NewPOS

3. POS Sends valid JSON to InVox Server

- POS sends the request to the **InVox Server with valid token**, which is responsible for handling transaction logic.

4. InVox Server Relays to MRA Server(This functionality is outside the scope of the NewPOS implementation and is managed by the InVox system)

- The InVox Server sends the transaction request to the **MRA Server**, which is likely the backend system responsible for processing the transaction.
- MRA may interact with databases, perform business logic, or validate the transaction.

5. MRA Server Processes and Responds(This functionality is outside the scope of the NewPOS implementation and is managed by the InVox system)

- The MRA Server processes the transaction (e.g., verifies payment, updates sales data).
- It generates a **response** indicating success or failure, along with any relevant data (e.g., authorization code, receipt info).
- This response is sent back to the InVox Server.

7. InVox Delivers Final Response to POS

- The InVox sends the final response back to the originating POS system.
- This completes the transaction communication loop.
- It may also log the success / Failures details.

8. POS Generates Receipt with QR Code

- Upon receiving a successful response, the POS system generates a **receipt**.
- The receipt includes transaction details and a **QR code on** footer section.
  Note: Failure to retrieve the QR code from the InVox Server does not prevent the fiscal transaction from being completed; NewPOS can continue processing.

Configuration Attributes	Required configurations:

```
<Section name="FiscalDbConfig">
<Parameter name="Enabled" value="true"/>
</Section>
```

```xml
<Section name="FiscalPluginConfig">
    <Parameter name="Enabled" value="true"/>
    <Parameter name="FiscalExtension" value="FiscalDataSignETA;FiscalUniqueInvoiceNumber"/>
    <Parameter name="FiscalExtensionSection" value="FiscalDataSignETA;FiscalUniqueInvoiceNumber;FiscalDbConfig"/>
</Section>
```

```xml
<Section name="FiscalTaskConfig">
    <Parameter name="NodeName" value="" />
    <Parameter name="Enabled" value="WAY"/>
    <Parameter name="FiscalExtension" value="FiscalReportingETA"/>
    <Parameter name="FiscalExtensionSection" value="FiscalReportingETA"/>
</Section>
<Section name="FiscalReportingETA">
    <Parameter name="DailySaleReport" value="../Out/InVox/NP6Sale"/>
    <Parameter name="ReportingMonitorPendingInterval" value="d:/Newpos61/OUT/invoices"/>
    <Parameter name="ReportingMonitorPath" value="d:/Newpos61/OUT/invoices"/>
</Section>
<Section name="FiscalUniqueInvoiceNumber">
    <Parameter name="InvPayAtCounterUseTenderPosId" value="true"/>
    <Parameter name="Path_UniqueReceiptNO" value="..\out"/>
    <Parameter name="MOPEnable" value="true"/>
    <Parameter name="ResetDate_UniqueReceiptNO" value="0401"/>
    <Parameter name="RemoveDateFromUniqueReceiptNO" value="false"/>
    <Parameter name="UniqueReceiptNOWith2DigitsPosNumber" value="false"/>
    <Parameter name="Use_UniqueReceiptNO" value="true"/>
    <Parameter name="UniqueReceiptNO_Scramble" value="true"/>
    <Parameter name="ReturnScrambleAndStandardUIN" value="true"/>
    <Parameter name="EnableRefundVATRegulation" value="true"/>
    <Parameter name="EnableDateFormatYYYYMM" value="true"/>
</Section>
<Section name="FiscalDataSignETA">
     <Parameter name="InVoxAPIAgentBaseURL" value="BASE_URL"/>
     <Parameter name="InVoxAPIEndpoint" value="ENDPOINT"/>
     <Parameter name="AgentResponseTimeout" value="10"/>
      <Parameter name="AgentRetry" value="3"/>
      <Parameter name="AgentRetryInterval" value="2"/>
      <Parameter name="ReceiptFail" value="../Out/InVox/Failed"/>
     <Parameter name="ReceiptSuccess" value="../Out/InVox/Success"/>
</Section>

```

Use Case 4 (Not required)	NP6 to generate an previous Note Hash - **This will be handled at InVox and is included here for reference purposes only (NOT required from NewPOS just set JSON field with 0)**
Assumptions	It is assumed that the **Previous Invoice Hash** will be generated by the **Newpos/EBS** and will correspond to the **last successfully fiscalised invoice** of the same type—whether **STD (Standard), CRN (Credit Note).** This hash must be included when submitting the **next invoice of the same type** for fiscalisation, ensuring continuity and traceability in the invoice chain.
Steps to Follow:	**Previous Invoice Hash – Generation Process** The **Previous Invoice Hash** is a digital fingerprint used to ensure the integrity and traceability of invoice data. It is generated using a **standardized hashing algorithm** (typically SHA-256) and includes key attributes from the **last fiscalised invoice of the same type**. **Hash Composition:** The hash should be generated using the following fields from the previous invoice: • **Date & Time** of the invoice • **Invoice Amount** • **Business Registration Number (BRN)** • **Invoice Number** These values are concatenated in a defined format and then hashed using the algorithm to produce a unique fingerprint. **Special Case – First Invoice:** For the **first invoice** of any type (e.g., STD, CRN), the value of `PreviousInvoiceHash` should be set to `0`. **Purpose:** This hash ensures:

<table>
<tr><td></td><td>

- The invoice chain is **tamper-proof**
- Each invoice is **cryptographically linked** to the previous one
- Compliance with **MRA's fiscalisation requirements**

</td></tr>
<tr><td>Samples:</td><td></td></tr>
<tr><td>User Interface</td><td></td></tr>
<tr><td>Technical Interface</td><td>

The `previousNoteHash` is a **hexadecimal hash** used to ensure invoice integrity. It is generated by hashing a concatenated string of:

- **Date & Time previous invoice issued equivalent** to dateTime
- **Invoice Amount equivalent to totalAmtPaid**
- **BRN equivalent** to brn (Seller)
- **Invoice Number equivalent** to invoiceIdentifier.

Field description	Field name
Date time previous invoice was issued	dateTime
Previous invoice amount	totalAmtPaid
BRN	brn
Previous invoice number	invoiceIdentifier

For example when generating an invoice, if previous invoice contains below values:

Field Name	Field Value
dateTime	20231019 14:54:51
totalAmtPaid	1000
brn	I2365XXXX
invoiceIdentifier	AINV101

The concatenated value should be as below:

Concatenated value
20231019 14:54:511000I2365XXXXAINV101

The previousNoteHash should contain below value after hashing above concatenated value:

Field previousNoteHash value
A78C2C5C5C3E33F1B4808D437F84BB303E0832D07A49912466EAF7137DF31EDC

Steps for Hashing Process

1. **Concatenate Fields**
   Combine the required fields in the specified order as described in the implementation guidelines.
2. **Apply SHA-256 Algorithm**
   Use the **SHA-256** hashing algorithm to generate a secure hash.
3. **Generate Byte Array**
   Compute the hash, which will produce an **array of bytes** as the output.
4. **Convert to Hexadecimal**
   Transform the byte array into its **hexadecimal string representation** for storage or transmission.

</td></tr>
<tr><td>Configuration Attributes</td><td></td></tr>
</table>

Use Case 5	**Seller Details to be filled in JSON**
Assumptions	NewPOS is functioning as expected and there are no network connectivity issues between the NewPOS system and the InVox server.
Steps to Follow:	NewPOS must retrieve the configuration settings defined in the `STOREDB/POSDB` XML files.
Samples:	
User Interface	
Technical Interface	**BTA recommends** introducing a **new parameter** in **StoreDB/POSDB** to capture seller-specific details, including: • `sellerName` • `sellerTAN`

- `sellerBRN`
- `sellerBusinessAddress`

Existing Configuration (Not Recommended for Reuse):

Reusing the existing configuration was **not recommended** due to its **loosely coupled structure**.

However, for reference, the current mappings are:

```
<StoreProfile>
    <TaxDefinition>
        <LegalName>Tribeca (DT)</LegalName>

<DefaultReceiptHeader>Welcome to McDonald's Mauritius\nStore ID: 3640015\nTel:471 3126\nemail:
info@mcdonalds.mu\nVAT No: VAT20183406\nBRN No: C07021843</DefaultReceiptHeader>
```

-
- `sellerName` → mapped to **LegalName**
- `sellerTAN` → mapped to **VAT No: VAT20183406**
- `sellerBRN` → mapped to **BRN No: C07021843**
- `sellerBusinessAddress` → proposed to be mapped to a **new element** to be added by **RFM** as this is a **Mandatory**, including:
  - Street Name
  - City
  - State
  - Postal Code

Configuration Attributes	**If the developer chooses to follow the recommendation from BTA, they should create new parameters under the following configuration path:** `StoreDB → <Configurations> → <Configuration type="Store.wide"> → <Section name="Receipt">` **Otherwise**, use the **existing configuration**, specifically the `<LegalName>` and `<DefaultReceiptHeader>` elements from **StoreDB**. **Note**: Complete configuration Setup will be found in the Release notes only.

Use Case 6	Tax code mapping
Assumptions	NewPOS is functioning as expected and there are no network connectivity issues between the NewPOS system and the InVox server.

Steps to Follow:	1. **NewPOS** must retrieve configuration settings defined in the **STOREDB/POSDB XML** files.
	2. According to the documentation provided by **Rogers Capital**, the `taxCode` attribute in the JSON payload is defined as follows:
	This attribute specifies the category of supplies made by the supplier. The possible values include:
	◦ **TC01**: Taxable standard supplies at 15%
	◦ **TC02**: Taxable standard supplies at zero rate
	◦ **TC03**: Exempt supplies
	◦ **TC04**: Non-fiscal items not affecting turnover
	◦ **TC05**: Standard-rated supplies made to exempt bodies or persons
	◦ **TC06**: Supplies outside the scope of VAT
	On the NewPOS side, we maintain the tax table in StoreDB as follows:
	`<TaxTable>`
	`  <TaxType statusCode="ACTIVE">`
	`  <TaxId>1</TaxId>`
	`  <TaxDescription>Mauritius Tax</TaxDescription>`

```
        <TaxRate>15</TaxRate>
        <TaxBasis>GROSS_AMOUNT</TaxBasis>
        <TaxCalcType>IN_HUNDRED</TaxCalcType>
        <Rounding>ROUND_DOWN</Rounding>
        <Precision>4</Precision>
    </TaxType>
    <TaxChain statusCode="ACTIVE">
        <TaxChainId>1</TaxChainId>
        <Rule>PARALLEL</Rule>
        <TaxTypeId taxId="1"/>
    </TaxChain>
</TaxTable>
```

Note: Currently, we have observed that only one TAX ID is configured in POSDATA. In future, market has extended TAXID's then market need to update new parameter TaxCodeMapping.

Samples:	Sample config as like <Parameter name="TaxCodeMapping" value="1:**TC01**"/>
User Interface	

Technical Interface	NewPOS must **read the newly introduced configuration attribute** and **assign its value to the corresponding field in the JSON payload**.
	
	The configuration should be defined in the following path:
	
	`StoreDB → <Configurations> → <Configuration type="Store.wide"> → <Section name="Receipt">`
	
	Add the parameter as :
	
	`<Parameter name="TaxCodeMapping" value="[NewPOS_TAX code]:[Rogers Capital Tax code label]"/>`
	
	NewPOS must apply the tax code mapping logic dynamically. This means that for each item in the sales view, the system should evaluate the relevant data and apply the appropriate mapping based on the configuration.
	
	`<taxChain>`
	`  <tax id="1" rate="15" type="TAX" baseAmt="117.39" amount="17.61"/>`
	`  </taxChain>`
	
	Explanation: Based on the sample provided:
	
	• If an item has **Tax ID = 1** and a **tax rate of 15**, then according to the configuration, the JSON payload should include:
	  ◦ "taxCodeMra": "TC01"
	• If an item has **Tax ID = 2** and a **tax rate of 0**, then the configuration parameter should be defined as:
	  ◦ <Parameter name="TaxCodeMapping" value="2:TC02"/>
	  ◦ In this case, NewPOS will populate the JSON payload with:
	  ▪ "taxCodeMra": "TC02"
	
	This logic ensures that tax codes are dynamically mapped based on the configuration and the tax details of each item in the sales view.

Configuration Attributes	Sample new config would be `StoreDB → <Configurations> → <Configuration type="Store.wide"> → <Section name="Receipt">` `<Parameter name="TaxCodeMapping" value="[NewPOS_TAX code]:[Rogers Capital Tax code label]"/>` **Sample config**: <Parameter name="TaxCodeMapping" value="1:**TC01**"/> **Note**: Complete configuration Setup will be found in the Release notes only.

Use Case 7	BR-4 **Robust Error Handling and User Alerts**
Assumptions	NewPOS Unable to connect to InVox server because of: Validation failure

	Network connectivity failure QR code not received from the InVox server
Steps to Follow:	**Validation Failure:** If the InVox server returns a validation failure (e.g. failure to get new token , invalid transaction data, server error), the POS system: • Displays an error message to the cashier indicating the issue. • Allows the cashier to retry the transaction or cancel it. • Logs the failure for audit or troubleshooting purposes. **Network Connectivity Issue:** If the POS system cannot connect to the InVox server due to network issues: • The system alerts the cashier about the connectivity problem. • The transaction proceeds with manual override. • The transaction is flagged for later validation once the connection is restored. **QR Code Generation Error:** If QR code generation fails after successful validation: • The system notifies the cashier of the error. • The receipt is printed without the QR code, or the QR code is generated manually if possible. • The transaction details are stored, and manual case handling is initiated.
Samples:	
User Interface	A standard Popup UI with Error message with next actions defined: • Popup Window title as Status Message as ERROR • Popup Body contains Code and Description of this message • Two action buttons OK/Retry, and Cancel
Technical Interface	NewPOS must prepare a standard popup message when an error receive. The error JSON sample would looks like below: `{ "status": "ERROR", "errorMessages": [ { "code": "ERR0020", "description": "Missing header parameters" } ] }` Every Error Message has the Status as ERROR with Code and Description. Please refer to list of error codes at SIP-55 - [MU] E-invoicing Implementation | Error Codes and Information: **Retry Mechanism for Network Issues** When NewPOS does not receive either a success or failure response, the situation is treated as a **network issue between NewPOS and the InVox server**. **Expected Behavior:** • Implement a **retry mechanism** for sending the JSON payload. • If the crew selects **"Retry"**, NewPOS should resend the API request immediately. • If the crew explicitly "**Cancel"** button pressed Then the JSON should be moved to the **"Unprocessed" folder**, so it can be sent later through an **asynchronous process** once connectivity is restored. The unprocessed folder mechanism will follow the same as other markets using e-invoicing (e.g. resending occurs every 24hrs). **INVOX Timeout Handling** If a timeout occurs before receiving a response from INVOX, a separate **GET endpoint** is available to **query the status** by passing the `invoiceIdentifier` as a parameter.

	This allows for asynchronous tracking and ensures that the invoice status can still be retrieved even if the initial request times out.
Configuration Attributes	

Use Case 8	Refund adjustments in JSON
Assumptions	
Steps to Follow:	**Refund by Order – Process Steps** 1. **Guideline Requirement** ◦ As per Rogers Capital's compliance, **NewPOS must include the original invoice number** in refund transactions to maintain proper reference. 2. **Market Adoption** ◦ To support this, markets must **adopt the "Refund by Order" feature** in NewPOS. 3. **Customer Interaction** ◦ The customer presents the **original order receipt** to the crew. 4. **Initiating Refund** ◦ The crew selects the **"Refund by Order"** option on the POS interface. 5. **Order Lookup** ◦ The crew enters either: ▪ **Order ID**, or ▪ Uses **search parameters** such as: • POS terminal ID • Transaction date • Order amount ◦ The matching order details are displayed on the POS screen. 6. **Refund Confirmation** ◦ The crew presses the **OK** button to confirm the refund. ◦ NewPOS finalizes the refund using either: ▪ **Cash** ▪ **Cashless** (e.g., card, digital wallet) 7. **Data Transmission** ◦ Once the refund is processed, **NewPOS prepares a JSON payload** including the original invoice number and refund details. ◦ This payload is sent to the **Invox Server**. 8. **QR Code Generation** ◦ The Invox Server processes the request and returns a **QR code** in the response. 9. **Receipt Printing** ◦ The **QR code is printed** on the **refund receipt** via the POS-connected printer.
Samples:	
User Interface	

Technical Interface	NewPOS must send API request as like USECASE 2 with change in URL and body parameters. NewPOS will have a refund view with **transactionKind** = 1 this means the current View is refund view otherwise 0 for Sale. Please review below list of **transactionKind** will represent the sale category.
	`Transaction kind, defines the mode this order representation is operating:`
	
	`0 - Transaction is a Sale`
	`1 - Transaction is a Refund`

```
2 - Transaction is a Waste
3 - Transaction is a Manager Discount
4 - Transaction is a Crew Discount
5 - Transaction is a Generic Discount
6 - Transaction is a Promotion
7 - Transaction is a Cashless Refund
8 - Transaction is a Giftcard Cashout
9 - Transaction is a Skip Car
10 - Transaction is a Billable Refund (only GC)
11 - Transaction is a Billable Refund (GC+PROD)
```

JSON Template Changes for Refunds

- **Seller Identification**:
  Remains **unchanged** from the original sale transaction.
- **Buyer Identification**:
  Not required, as this is a **B2C transaction**.
- **Invoice Header Modifications**: Along with other JSON fields below fields must be changed for refund transaction
  - `invoiceTypeMra` : Must be set to `"CRN"` (Credit Note).
  - `invoiceRefIdentifier` : Must be populated with the **original sale's** `invoiceIdentifier` to maintain reference linkage.
  - `reasonStated` : Must be filled with a reason for refund.
  - `salesTransactions` : Must be set as " `CREDIT` ".
- **Invoice Line Items**:
  - The **description of goods/services** should be the **same as in the original sale**.

Please refer to the JSON template for more details: SIP-55 - [MU] E-invoicing Implementation | JSON Template to be prepared from NewPOS

Note: Please refer to InVox Server requests and response samples in Use Case 2.

Use Case 9	salesTransactions mapping
Assumptions	NewPOS is functioning as expected and there are no network connectivity issues between the NewPOS system and the InVox server.
Steps to Follow:	1. **NewPOS** must retrieve configuration settings defined in the **STOREDB/POSDB XML** files. 2. According to the documentation provided by **Rogers Capital**, the salesTransactions attribute in the JSON payload is defined as follows: The possible list of values are a. CASH = Cash a. CASH = Cash b. BNKTRANSFER = Bank Transfers c. CHEQUE = Cheques d. CARD = Cards e. OTHER b. CREDIT = Credit On the NewPOS side, we maintain the Tender table in StoreDB as follows:

```xml
<TenderTypes>
    <TenderType statusCode="ACTIVE">
      <TenderId>0</TenderId>
      <TenderFiscalIndex>0</TenderFiscalIndex>
      <TenderName>Cash Rs</TenderName>
      <TenderChange type="CHANGE_WITH" roundToMinAmount="true" maxAllowed="2000" />
      <TenderCategory>TENDER_NATIVE</TenderCategory>
      <TaxOption>CALCULATE</TaxOption>
      <SubtotalOption>CALCULATE</SubtotalOption>
      <CurrencyDecimals>0</CurrencyDecimals>
      <TenderRoundingRule>FALSE</TenderRoundingRule>
```

```
    </TenderType>
    <TenderType statusCode="ACTIVE">
      <TenderId>10</TenderId>
      <TenderName>Cashless Common</TenderName>
      <TenderChange type="CHANGE_WITH" roundToMinAmount="true" />
      <TenderCategory>TENDER_ELECTRONIC_PAYMENT</TenderCategory>
      <TenderFlags>
        <TenderFlag>NOT_OPEN_DRAWER</TenderFlag>
        <TenderFlag>NO_SKIM</TenderFlag>
      </TenderFlags>
      <TaxOption>CALCULATE</TaxOption>
      <SubtotalOption>CALCULATE</SubtotalOption>
      <CurrencyDecimals>0</CurrencyDecimals>
      <ElectronicPayment>
        <LegacyId>EFTPOS0</LegacyId>
      </ElectronicPayment>
      <TenderRoundingRule>FALSE</TenderRoundingRule>
    </TenderType>
    <TenderType statusCode="ACTIVE">
      <TenderId>11</TenderId>
      <TenderName>Offline Cashless</TenderName>
      <TenderChange id="21" type="CHANGE_WITH" roundToMinAmount="false" maxAllowed="0" />
      <TenderCategory>TENDER_ELECTRONIC_PAYMENT</TenderCategory>
      <TenderFlags>
        <TenderFlag>NOT_OPEN_DRAWER</TenderFlag>
        <TenderFlag>NO_SKIM</TenderFlag>
      </TenderFlags>
      <TaxOption>CALCULATE</TaxOption>
      <SubtotalOption>CALCULATE</SubtotalOption>
      <CurrencyDecimals>2</CurrencyDecimals>
      <ElectronicPayment>
        <LegacyId>EFTP99</LegacyId>
      </ElectronicPayment>
    </TenderType>
    <TenderType statusCode="ACTIVE">
      <TenderId>12</TenderId>
      <TenderName>Cashless Visa</TenderName>
      <TenderChange type="CHANGE_WITH" roundToMinAmount="true" />
      <TenderCategory>TENDER_ELECTRONIC_PAYMENT</TenderCategory>
      <TenderFlags>
        <TenderFlag>NOT_OPEN_DRAWER</TenderFlag>
        <TenderFlag>NO_SKIM</TenderFlag>
      </TenderFlags>
      <TaxOption>CALCULATE</TaxOption>
      <SubtotalOption>CALCULATE</SubtotalOption>
      <CurrencyDecimals>0</CurrencyDecimals>
      <ElectronicPayment>
        <LegacyId>EFTPOS1</LegacyId>
      </ElectronicPayment>
    </TenderType>
    <TenderType statusCode="ACTIVE">
      <TenderId>13</TenderId>
      <TenderName>Cashless MasterCard</TenderName>
      <TenderChange type="CHANGE_WITH" roundToMinAmount="true" />
      <TenderCategory>TENDER_ELECTRONIC_PAYMENT</TenderCategory>
      <TenderFlags>
        <TenderFlag>NOT_OPEN_DRAWER</TenderFlag>
        <TenderFlag>NO_SKIM</TenderFlag>
      </TenderFlags>
      <TaxOption>CALCULATE</TaxOption>
      <SubtotalOption>CALCULATE</SubtotalOption>
      <CurrencyDecimals>0</CurrencyDecimals>
      <ElectronicPayment>
        <LegacyId>EFTPOS2</LegacyId>
      </ElectronicPayment>
    </TenderType>
    <TenderType statusCode="ACTIVE">
      <TenderId>16</TenderId>
      <TenderName>USD</TenderName>
      <TenderChange type="CHANGE_WITH" roundToMinAmount="true" />
      <TenderCategory>TENDER_FOREIGN_CURRENCY</TenderCategory>
      <TaxOption>CALCULATE</TaxOption>
      <SubtotalOption>CALCULATE</SubtotalOption>
      <CurrencyDecimals>0</CurrencyDecimals>
      <ForeignCurrency>
        <LegacyId>16</LegacyId>
        <ExchangeRate>28</ExchangeRate>
        <Orientation>FOREIGN_NATIVE</Orientation>
        <Precision>XX.XXYY</Precision>
        <Rounding>ROUND_UP</Rounding>
        <ExchangeMode>BY_TOTAL</ExchangeMode>
      </ForeignCurrency>
    </TenderType>
    <TenderType statusCode="ACTIVE">
      <TenderId>17</TenderId>
```

```xml
        <TenderName>Euro</TenderName>
        <TenderChange type="CHANGE_WITH" roundToMinAmount="true" />
        <TenderCategory>TENDER_FOREIGN_CURRENCY</TenderCategory>
        <TaxOption>CALCULATE</TaxOption>
        <SubtotalOption>CALCULATE</SubtotalOption>
        <CurrencyDecimals>0</CurrencyDecimals>
        <ForeignCurrency>
          <LegacyId>17</LegacyId>
          <ExchangeRate>38</ExchangeRate>
          <Orientation>FOREIGN_NATIVE</Orientation>
          <Precision>XX.XXYY</Precision>
          <Rounding>ROUND_UP</Rounding>
          <ExchangeMode>BY_TOTAL</ExchangeMode>
        </ForeignCurrency>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>21</TenderId>
        <TenderName>Credit Sales</TenderName>
        <TenderChange type="EXACT_AMOUNT" roundToMinAmount="false" />
        <TenderCategory>TENDER_CREDIT_SALES</TenderCategory>
        <TenderFlags>
          <TenderFlag>NOT_OPEN_DRAWER</TenderFlag>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>CALCULATE</TaxOption>
        <SubtotalOption>CALCULATE</SubtotalOption>
        <CurrencyDecimals>0</CurrencyDecimals>
        <CreditSales>
          <LegacyId>21</LegacyId>
        </CreditSales>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>22</TenderId>
        <TenderName>MCB Cashless</TenderName>
        <TenderChange id="0" type="CHANGE_WITH" roundToMinAmount="true" />
        <TenderCategory>TENDER_ELECTRONIC_PAYMENT</TenderCategory>
        <TenderFlags>
          <TenderFlag>NOT_OPEN_DRAWER</TenderFlag>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>CALCULATE</TaxOption>
        <SubtotalOption>CALCULATE</SubtotalOption>
        <CurrencyDecimals>0</CurrencyDecimals>
        <ElectronicPayment>
          <LegacyId>EFTPOS10</LegacyId>
        </ElectronicPayment>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>23</TenderId>
        <TenderName>MYT Money</TenderName>
        <TenderChange type="EXACT_AMOUNT" roundToMinAmount="false" />
        <TenderCategory>TENDER_CREDIT_SALES</TenderCategory>
        <TenderFlags>
          <TenderFlag>NOT_OPEN_DRAWER</TenderFlag>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>CALCULATE</TaxOption>
        <SubtotalOption>CALCULATE</SubtotalOption>
        <CurrencyDecimals>0</CurrencyDecimals>
        <CreditSales>
          <LegacyId>23</LegacyId>
        </CreditSales>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>24</TenderId>
        <TenderName>POP</TenderName>
        <TenderChange type="EXACT_AMOUNT" roundToMinAmount="false" />
        <TenderCategory>TENDER_CREDIT_SALES</TenderCategory>
        <TenderFlags>
          <TenderFlag>NOT_OPEN_DRAWER</TenderFlag>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>CALCULATE</TaxOption>
        <SubtotalOption>CALCULATE</SubtotalOption>
        <CurrencyDecimals>0</CurrencyDecimals>
        <CreditSales>
          <LegacyId>24</LegacyId>
        </CreditSales>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>25</TenderId>
        <TenderName>MCB WIRELESS</TenderName>
        <TenderChange type="EXACT_AMOUNT" roundToMinAmount="false" />
        <TenderCategory>TENDER_CREDIT_SALES</TenderCategory>
        <TenderFlags>
```

```
          <TenderFlag>NOT_OPEN_DRAWER</TenderFlag>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>CALCULATE</TaxOption>
        <SubtotalOption>CALCULATE</SubtotalOption>
        <CurrencyDecimals>0</CurrencyDecimals>
        <CreditSales>
          <LegacyId>25</LegacyId>
        </CreditSales>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>26</TenderId>
        <TenderName>Bank Cheque</TenderName>
        <TenderChange type="EXACT_AMOUNT" roundToMinAmount="false" />
        <TenderCategory>TENDER_CREDIT_SALES</TenderCategory>
        <TenderFlags>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>CALCULATE</TaxOption>
        <SubtotalOption>CALCULATE</SubtotalOption>
        <CurrencyDecimals>0</CurrencyDecimals>
        <CreditSales>
          <LegacyId>26</LegacyId>
        </CreditSales>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>30</TenderId>
        <TenderFiscalIndex>30</TenderFiscalIndex>
        <TenderName>MyOrder</TenderName>
        <TenderChange type="EXACT_AMOUNT" roundToMinAmount="false" />
        <TenderCategory>TENDER_OTHER_PAYMENT</TenderCategory>
        <TenderFlags>
          <TenderFlag>NOT_OPEN_DRAWER</TenderFlag>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>CALCULATE</TaxOption>
        <SubtotalOption>CALCULATE</SubtotalOption>
        <DefaultSkimLimit>0</DefaultSkimLimit>
        <DefaultHaloLimit>0</DefaultHaloLimit>
        <CurrencyDecimals>2</CurrencyDecimals>
        <OtherPayment>
          <LegacyId>MyOrder</LegacyId>
        </OtherPayment>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>31</TenderId>
        <TenderFiscalIndex>31</TenderFiscalIndex>
        <TenderName>MDSC Order</TenderName>
        <TenderChange type="EXACT_AMOUNT" roundToMinAmount="false" />
        <TenderCategory>TENDER_OTHER_PAYMENT</TenderCategory>
        <TenderFlags>
          <TenderFlag>NOT_OPEN_DRAWER</TenderFlag>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>CALCULATE</TaxOption>
        <SubtotalOption>CALCULATE</SubtotalOption>
        <DefaultSkimLimit>0</DefaultSkimLimit>
        <DefaultHaloLimit>0</DefaultHaloLimit>
        <CurrencyDecimals>2</CurrencyDecimals>
        <OtherPayment>
          <LegacyId>MDSC Order</LegacyId>
        </OtherPayment>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>1</TenderId>
        <TenderFiscalIndex>1</TenderFiscalIndex>
        <TenderName>Coupon A</TenderName>
        <TenderChange type="KEEP_EXCESS" roundToMinAmount="false" />
        <TenderCategory>TENDER_GIFT_COUPON</TenderCategory>
        <TenderFlags>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>REDUCE</TaxOption>
        <SubtotalOption>REDUCE</SubtotalOption>
        <CurrencyDecimals>2</CurrencyDecimals>
        <GiftCoupon>
          <LegacyId>Coupon A</LegacyId>
          <Amount>100</Amount>
        </GiftCoupon>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>2</TenderId>
        <TenderFiscalIndex>2</TenderFiscalIndex>
        <TenderName>Coupon B</TenderName>
        <TenderChange type="KEEP_EXCESS" roundToMinAmount="false" />
```

```
        <TenderCategory>TENDER_GIFT_COUPON</TenderCategory>
        <TenderFlags>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>REDUCE</TaxOption>
        <SubtotalOption>REDUCE</SubtotalOption>
        <CurrencyDecimals>2</CurrencyDecimals>
        <GiftCoupon>
          <LegacyId>Coupon B</LegacyId>
          <Amount>250</Amount>
        </GiftCoupon>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>3</TenderId>
        <TenderFiscalIndex>3</TenderFiscalIndex>
        <TenderName>Coupon C</TenderName>
        <TenderChange type="KEEP_EXCESS" roundToMinAmount="false" />
        <TenderCategory>TENDER_GIFT_COUPON</TenderCategory>
        <TenderFlags>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>REDUCE</TaxOption>
        <SubtotalOption>REDUCE</SubtotalOption>
        <CurrencyDecimals>2</CurrencyDecimals>
        <GiftCoupon>
          <LegacyId>Coupon C</LegacyId>
          <Amount>500</Amount>
        </GiftCoupon>
      </TenderType>
      <TenderType statusCode="ACTIVE">
        <TenderId>5</TenderId>
        <TenderFiscalIndex>5</TenderFiscalIndex>
        <TenderName>Coupon E (Lucky.mu)</TenderName>
        <TenderChange type="KEEP_EXCESS" roundToMinAmount="false" />
        <TenderCategory>TENDER_GIFT_COUPON</TenderCategory>
        <TenderFlags>
          <TenderFlag>NO_SKIM</TenderFlag>
        </TenderFlags>
        <TaxOption>REDUCE</TaxOption>
        <SubtotalOption>REDUCE</SubtotalOption>
        <CurrencyDecimals>2</CurrencyDecimals>
        <GiftCoupon>
          <LegacyId>Coupon E</LegacyId>
          <Amount>1000</Amount>
        </GiftCoupon>
      </TenderType>
    </TenderTypes>
```

Samples:	Sample config as like <Parameter name="TenderIDMappingToInVOX" value="CASH:**0**|BNKTRANSFER:|CHEQUES:26|CARDS:10,11,12,13,22|OTHERS:[Remaining ALL TenderIDS]"/>
User Interface	
Technical Interface	NewPOS must dynamically apply Tender ID mappings based on a newly introduced configuration attribute. This configuration enables the system to correctly associate tender types with their corresponding IDs during the payment completion process. The configuration should be defined in the following path:

```

<Configurations>
  <Configuration type="Store.wide">
    <Section name="Receipt">
      <Parameter name="TenderIDMappingToInVOX" value="CASH:0|BNKTRANSFER:|CHEQUES:26|CARDS:10,11,12,13,22|OTHERS:[Remaining ALL TenderIDS]"/>
    </Section>
  </Configuration>
</Configurations>

```

Expected Behavior:

- **Read Configuration**:
  NewPOS must read the `TenderIDMappingToInVOX` parameter from the specified configuration path.
- **Assign to Payload**:
  The retrieved value should be assigned to the corresponding field in the JSON payload used during transaction processing.

- **Dynamic Mapping Logic**:
  During each sales transaction, when payment is being finalized, NewPOS should:
  - Identify the tender types used.
  - Evaluate the configuration mapping.
  - Apply the appropriate Tender ID(s) based on the mapping rules.

Handling Coupon and Foreign Tender Types in NewPOS (TenderIDMappingToInVOX→CASH:0)

- All **coupon-type** (discount-related) and **foreign-currency tender IDs** are generally treated as **cash transactions** within NewPOS.
- If a transaction is paid using a combination of **Coupon** and **Cash**, it should be classified as a **Cash** transaction overall.
- Similarly, if a transaction involves **Foreign currency** and **Cash**, the system must still treat it as a **Cash** transaction.
- However:
  - If a transaction includes a **coupon** and the remaining payment is made via **cashless methods** (e.g., cards), the transaction is classified as **Cards**.
  - Likewise, if a transaction is paid using **foreign currency** and **cashless methods**, the payment is treated as **Cards**.

In all these scenarios, the `salesTransactions` field in the JSON payload must be set to either **"CASH"** or **"CARDS"** to reflect the unified classification of the payment method.

Configuration Attributes

Sample new config would be `StoreDB → <Configurations> → <Configuration type="Store.wide"> → <Section name="Receipt">`

```
<Configurations>
  <Configuration type="Store.wide">
    <Section name="Receipt">
      <Parameter name="TenderIDMappingToInVOX" value="CASH:0|BNKTRANSFER:|CHEQUES:26|CARDS:10,11,12,13,22|OTHERS:[Remaining ALL TenderIDS]"/>
    </Section>
  </Configuration>
</Configurations>
```

Note: Complete configuration Setup will be found in the Release notes only.

JSON Template to be prepared from NewPOS

Components								
Seller Identification	**Item Description**	**Definitions**	**JSON element**	**Data Type**	**Size**	**Possible Values**	**Requirement**	**Mapping to NP6**

	name	This attribute defines the legal name of the person supplying the	seller Name	String	100	Rogers Capital Outsourcing Ltd	Mandatory	`<StoreProfile>`
								`<TaxDefinition>`
								`<LegalName>Tribeca (DT)</LegalName>`
								Note: **Please refer to use case 5**

	goods/services						
tradeName	This attribute defines the trade name of the person supplying the goods/services.	sellertradeName	String	100		Optional	As this option so make this empty.
tan	This attribute defines the VAT number of the person supplying the goods/services	seller TAN	String	8	20475358	Mandatory	<DefaultReceiptHeader>Welcome to McDonald's Mauritius\nStore ID: 3640015\nTel:4713126\nemail: info@mcdonalds.mu\n**VAT No: VAT20183406**\nBRN No: C07021843</DefaultReceiptHeader> → Map to **VAT No: VAT20183406** **Note**: **Please refer to use case 5**
brn	This attribute defines the Business Registration Number (BRN) of the person supplying the goods/services.	seller BRN	String	10	C08084517	Mandatory	<DefaultReceiptHeader>Welcome to McDonald's Mauritius\nStore ID: 3640015\nTel:4713126\nemail: info@mcdonalds.mu\nVAT No: VAT20183406\n**BRN No: C07021843**</DefaultReceiptHeader> → Mapped to **BRN No: C07021843** **Note: Please refer to use case 5**
businessAddr	This attribute allows the business address of the person supplying the goods/services.	seller BusinessAddress	String	250	Rogers House Port-Louis	Mandatory	**RFM change: they need to include Address**: <DefaultReceiptHeader>Welcome to McDonald's Mauritius\n**Address: Street name, City, State, Postal code**. \nStore ID: 3640015\nTel:4713126\nemail: info@mcdonalds.mu\nVAT No: VAT20183406\n**BRN No: C07021843**</DefaultReceiptHeader>

							→ Mapped to **Address: Street name, City, State, Postal code**. **Note: Please refer to use case 5**
businessPhoneNo	This attribute allows the telephone number of the person supplying the goods/services. Numbers allowed 0 to 9 and +. For example: +2302076000	sellerPhNo	String	20	2117801	Optional	
ebsCounterNo	This attribute defines the counter/terminal details (number, name) of the EBS where receipt was issued to buyer. This counter number is issued by the taxpayer. This should appear on the printed invoice	ebsCounterNo	String	20	10	Optional	This currently Available, This is POS register number. This should be mapped to tenderPOS="POS0011" from View XML (only the number)
Cashier ID	This attribute defines the ID of the cashier operating the EBS.	cashierID	String	100	4180	Optional	This is current cashier who sold this transaction to customer. This is also available in NewPOS session sate variable.

Buyer Identification	

Item Description	Description	JSON element	Data Type	Size	Possible Values	Requirement	Mapping to NP6
name	This attribute defines the legal name of the person to whom the goods / services are supplied This attribute is mandatory for B2B and B2G transactions.	buyerName	string	100	Rogers Capital Technology Services Ltd	conditional	**Default**: Empty As MCD Restaurants have only B2C type of transactions then B2B transactions are out of scoped.
buyerType	This attribute defines the category of the buyer. The possible values are 1. VATR = VAT Registered Person 2. NVTR = Non VAT Registered Person 3. EXMP = Exempt Person This attribute is mandatory for B2B and B2G transactions.	buyerTypeMra	string	4	VATR	Conditional	**Default**: NVTR As MCD Restaurants have only B2C type of transactions then B2B transactions are out of scoped.
tan	This attribute defines the VAT number of the person to whom the goods/services are supplied. This attribute is mandatory for B2B and B2G transactions.	buyerTAN	string	8	20216629	Conditional	**Default**: Empty As MCD Restaurants have only B2C type of transactions then B2B transacti

							ons are out of scoped.
businessAddr	The business address of the person to whom the goods/services are supplied.	buyerBusinessAddress	string	100	Rogers Capital Technology Services Ltd	optional	**Default**: Empty As MCD Restaurants have only B2C type of transactions then B2B transactions are out of scoped.
brn	The BRN of the person to whom the goods/services are supplied. This attribute is mandatory for B2B.	buyerBRN	string	10	C07027054	Conditional	**Default**: Empty As MCD Restaurants have only B2C type of transactions then B2B transactions are out of scoped.
nic	The National Identity Card of the buyer. May be implemented at a later stage.	-	string	14		optional	-Not required

Invoice Header

Item Description	Description	JSON element	Data Type	Size	Possible Values	Requirement	Mapping to NP6
personType	This attribute allows the seller to provide its VAT Registration Status. The possible values are 1. VATR = VAT Registered Person 2. NVTR = Non VAT Registered Person	personTypeMra	String	4	VATR	Mandatory	**Default** - NVTR As MCD Restaurants have only B2C type of transactions then B2B transactions are out of scoped.

transactionType	This attribute allows the seller to define the nature of transaction. The possible values are 1. B2B = Business to Business 2. B2G = Business to Government 3. B2C = Business to Consumer 4. EXP = Zero rated supplies – Exports 5. B2E = Taxable supplies made to exempt bodies or persons	transactionTypeMra	string	3	B2C	Mandatory	**Default** is B2C. As MCD Restaurants have only B2C type of transactions then B2B transactions are out of scoped.
invoiceTypeMra	This attribute allows the seller to define the type of invoice issued by the seller. The possible values are 1. STD = Standard 2. PRF = Proforma(Not supported) 3. TRN = Training (Not supported) 4. CRN = Credit 5. DRN = Debit (Not supported)	invoiceTypeMra	string	3	STD	Mandatory	**Default** : STD **SALE** → STD **Refund** → CRN Based on transactionKind from View we will maintain this feild
invoiceIdentifier	This attribute defines the sequential invoice number which is automatically generated by the EBS on the e-invoice and/or associated credit/debit notes.	invoiceIdentifier	string	100	INVOX0135	Mandatory	Unique Invoice feature to be used. this feature will generate invoice number automatically. This invoice is unique and maintained per register. Setup guide "https://wiki.rdisoftware.com/display/IOMBUIDL/Fiscal+Unique+Invoice+Number+Configuration" **Note**: We need to make sure that the receipt number does not reset during the same business date. Some configuration reset the invoice

							back to 1 once it reaches 100 orders.
invoiceCounter	The attribute defines the number of invoices generated by the EBS. The counter cannot be reset except that it may automatically reset to minimum value on reaching maximum counts.	invoiceCounter	string	20	1	Mandatory	W can use the order ID from view. → Mapped to orderId="06" Also we can refer to the the order ID printed on Sale/ Refund receipt. **Note**: We need to make sure that the receipt number does not reset during the same business date. Some configuration reset the invoice back to 1 once it reaches 100 orders.
invoiceRefIdentifier	This attribute defines the sequential number of an existing invoice generated by an EBS (invoiceIdentifier). The attribute is mandatory if invoiceTypeDesc is a CRN or a DRN.	invoiceRefIdentifier	string	100		Mandatory	Original **invoiceIdentifier** in case of refund transaction. For sale this will be empty.
previousNoteHash	This attribute defines the digital fingerprint of a document obtained using a standardized hashing algorithm to prevent modifying or tampering with the original data. Previous Invoice Hash will be based on last invoice for each type of invoice (STD, CRN, DRN, PRF & TRN) at the time when the next invoice is sent for fiscalisation and shall be generated by the EBS. Previous Invoice Hash = (Date & Time + Invoice Amount + BRN + invoice number) of previous invoice. Refer to Technical Guide to prepare and send EBS	previousNoteHash	string	100	0	Mandatory	Roger Capitals has confirmed that the `PreviousNoteHash` **field is managed by INVOX**. From the INVOX chain perspective, the data is processed correctly as it is received. Since Roger Capitals will continue to maintain this field, the **NewPOS system will consistently send a value of** `0` for `PreviousNoteHash`. This approach aligns

	transactions for details. 0 should be sent as value for first invoice.						with the current integration and ensures smooth processing without conflict.
dateTimeInvoiceIssued	This attribute defines the date and time the e-invoice was generated on the EBS. It is anticipated that this attribute will be automatically populated by the EBS. Format should be yyyyMMdd HH:mm:ss.	dateTimeInvoiceIssued	string	17	20240625 10:38:27	Mandatory	**Format**: YYYYMMDD HH:MM:SS saleDate="20251013" saleTime="200836" This is same as receipt date and time printed.
totalVatAmount	This attribute defines the total VAT amount collected for the purchase of goods/services. If purchase consists of non VATable items, totalVatAmount should be 0. Attribute should contain only numeric values and should be rounded to 2 decimal places.	totalVatAmount	string	20	2700	Mandatory	totalTax="24.78" Sample View shown below `<View taxMode="0" BPTotalAmount="190.00" BPGrossAmount="190.00" BPTotalTax="24.78" BDTotalAmount="190.00" BDGrossAmount="190.00" BDTotalTax="24.78" totalAmount="190.00" grossAmount="190.00" totalTax="24.78"`
totalAmtWoVatMur	This attribute defines the total amount without VAT for the purchase of goods/services expressed in Mauritian currency if invoice is issued in foreign currency.	**totalAmtWoVatMur**	string	20	27800	Mandatory	Difference between grossAmount="190.00" totalTax="24.78" `<View taxMode="0" BPTotalAmount="190.00" BPGrossAmount="190.00" BPTotalTax="24.78" BDTotalAmount="190.00" BDGrossAmount="`

								190.00" BDTotalTax="24.78" totalAmount="190.00" grossAmount="190.00" totalTax="24.78"
	totalAmtWoVatCur	This attribute defines the total amount without VAT for the purchase of goods/services. Attribute should contain only numeric values and should be rounded to 2 decimal places.	totalAmtWoVatCur	string	20	27800	Conditional	These fields represent the transaction amount excluding VAT: • `totalAmtWoVatCur`: Amount in the transaction's original (foreign) currency • `totalAmtWoVatMur`: Equivalent amount converted to Mauritian Rupees (MUR) **Behavior Based on Currency:** • If the transaction is in **MUR**, both fields will have the **same value**. • If the transaction is in a **foreign currency**: ◦ `totalAmtWoVatCur` will show the amount in the foreign currency. ◦ `totalAmtWoVatMur` will show the converted amount in MUR. **Examples:** • **Foreign Sale (10 USD):**

							◦ `totalAmtWoVatCur`: 10 USD ◦ `totalAmtWoVatMur`: Rs 45 • **Local Sale (Rs 45):** ◦ `totalAmtWoVatCur`: Rs 45 ◦ `totalAmtWoVatMur`: Rs 45
currency	This attribute defines the currency in which the invoice is issued. Default value should be MUR but should be open for other currencies	currency	string	3	MUR	Mandatory	Default MUR, USD and Euro are two possible foreign currencies used for this market.
noofitems	Number of Item Lines	noofitems	string	20	5	Mandatory	Total number of items in a sale or refund.
invoiceTotal	This attribute defines the total amount to be paid inclusive of VAT for the purchase of goods/services. invoiceTotal = totalVatAmount + totalAmtWoVatCur	invoiceTotal	string	20	27800	Mandatory	totalAmount="190.00" `<View taxMode="0" BPTotalAmount="190.00" BPGrossAmount="190.00" BPTotalTax="24.78" BDTotalAmount="190.00" BDGrossAmount="190.00" BDTotalTax="24.78" totalAmount="190.00" grossAmount="190.00" totalTax="24.78"`
discountTotalAmount	This attribute defines the discount/rebate amount if present for total sales.	discountTotalAmount	string	20	1000	Conditional	If this is Promo then TrnsactionKind = 6 (Promotion/ Promo) Then from the View `<View taxMode="0"`

							BPGrossAmount="210.00" grossAmount="110.00" **Formula**: discountTotalAmount = BPGrossAmount - grossAmount Otherwise for transactionKind= 3(Manger Meal Discount),4(Crew Meal Discount),5(Generic Discounts) <View taxMode="0" BDGrossAmount="210.00" grossAmount="110.00" **Formula**: discountTotalAmount = BDGrossAmount-grossAmount
totalAmtPaid	This attribute defines the total amount paid inclusive of VAT for the purchase of goods/services. totalAmtPaid = invoiceTotal - downPayment - discountTotalAmount Attribute should contain only numeric values and should be rounded to 2 decimal places.	totalAmountPaid	string	20	30500	Mandatory	totalAmount="190.00" <View taxMode="0" BPTotalAmount="190.00" BPGrossAmount="190.00" BPTotalTax="24.78" BDTotalAmount="190.00" BDGrossAmount="190.00" BDTotalTax="24.78" totalAmount="190.00" grossAmount="190.00" totalTax="24.78"
reasonStated	This attribute allows seller to define the reason for issuance of credit/debit note. This attribute is mandatory if	reasonStated	string	100	refund reason	Conditional	**Default**: Customer Requested for Refund.

	invoiceTypeDesc is either CRN or DRN.						
salesTransactions	This attributes allows seller to define whether the sales transaction was done in Cash or Credit. The possible list of values are 1. CASH = Cash a. CASH = Cash b. BNKTRANSFER = Bank Transfers c. CHEQUE = Cheques d. CARD = Cards e. OTHER 2. CREDIT = Credit	salesTransactions	string	6	CASH	Mandatory	**Default**: CASH Sale always CASH, however we do have different modes of cash payment payments. By Default Refunds are **CREDIT** **Please refer to USE CASE 9**

Description of Goods/Services/Invoice Line

mrainvoicelines": [{item1, item2,...}]

Item Description	Definitions	JSON element	Data Type	Size	Possible Values	Requirement	Mapping to NP6
itemNo	This attribute defines the line item number	itemNo	String	10	1	Mandatory	Mapp to View→ ItemView→ <itemCode>2</itemCode `<View taxMode="0" <ItemView> <itemCode>0</itemCode>`
taxCode	This attribute defines the category of supplies made by the supplier. The possible list of values are 1. TC01 = Taxable SS at rate of 15% 2. TC02 = Taxable SS at zero rate 3. TC03 = Exempt SS 4. TC04 = Non fiscal items not affecting turnover 5. TC05: Standard rated supplies made to exempt bodies or persons 6. TC06: Supplies outside scope of VAT	taxCodeMra	String	4	TC01	Mandatory	New change to map NewPOS tax ID vs with New tax codes as per MRA guidelines. Please refer to **Use Case 6**

nature	This attribute defines the nature of supplies made to buyer. The possible list of values are 1. GOODS = Goods 2. SERVICES = Services	nature Mra	String	8	GOODS	Mandatory	Default Goods. This depends on product code **productCategory** either FOOD or Service If **productCategory** = FOOD then Make it as GOODS otherwise SERVICES .
itemDesc	The description of goods/services are supplied	itemDesc	String	100	BIG Mac	Mandatory	Map to View → ItemView → <longName>Value Chicken</longName> `<View taxMode="0" <ItemView> <longName>Value Chicken</longName>`
productCodeMra	May be published on MRA website. Leave blank for the time being.	productCodeMRA	String	8			-
productCodeOwn	As per supplier's own records	productCodeOwn	String	100	9651	optional	`<View taxMode="0" <ItemView> <productCode>1001</productCode>`
unitPrice	This attribute defines the unit price of goods supplied. This attribute is mandatory if "Nature of Supplies"= Goods	unitPrice	String	20	2100	Conditional	`<View taxMode="0" <ItemView> <unitPrice>75.00 </unitPrice>`
quantity	This attribute defines the quantity of goods supplied. This attribute is mandatory if "Nature of Supplies"= Goods	quantity	String	20	2	Conditional	`<View taxMode="0" <ItemView> <quantity>1</quantity>`
discount	This attribute defines the discount/rebate amount if present for the sale of item.	discount	String	20	1000	Conditional	`<View taxMode="0" <ItemView> <taxChain> <tax id="1" rate="15" type="TAX" baseAmt="65.22" amount="9.78" ADBaseAmt="34.1`

								7" ADAmount="5.12" /> This calculation should only be performed **when the** `<discount>1</discount>` **tag is present** within the `ItemView`. **Calculation Formula:** (baseAmt+amount) – (ADBaseAmt+ADAmount) This logic ensures that the discount-related adjustment is applied only when explicitly indicated by the discount tag.
		discountedValue	This attribute defines the amount after discount. The amount is exclusive of VAT.	discountedValue	String	20	15000	Conditional	<View taxMode="0" <ItemView> <taxChain> <tax id="1" rate="15" type="TAX" baseAmt="65.22" amount="9.78" ADBaseAmt="34.17" ADAmount="5.12" /> This calculation should only be performed **when the** `<discount>1</discount>` **tag is present** within the `ItemView`. **Calculation Formula:**

							(baseAmt)–(ADBaseAmt) This logic ensures that the discount-related adjustment is applied only when explicitly indicated by the discount tag.
amtWoVatCur	This attribute defines the amount without VAT for the purchase of goods/services.	amtWoVatCur	String	20	35000	Mandatory	The fields `AmtWoVatCur` and `AmtWoVatMur` represent transaction amounts **excluding VAT**, in **foreign currency** and **Mauritian Rupees (MUR)** respectively. • If the transaction is in **MUR**, both fields will have the **same value**. • If the transaction is in a **foreign currency**, then: ◦ `AmtWoVatCur` will reflect the amount in the **foreign currency** ◦ `AmtWoVatMur` will show the **converted equivalent in MUR**
amtWoVatMur	This attribute defines the amount without VAT for the purchase of goods/services expressed in Mauritian currency if invoice is issued in foreign currency. It should be numeric.	amtWoVatMur	String	20	17000	Conditional	`<View taxMode="0" <ItemView> <netTotalPrice>65.22</netTotalPrice>`
vatAmt	This attribute allows the seller to specify the VAT amount collected on the VATable item. vatAmt should be numeric and	vatAmount	String	20	12000	Mandatory	Only if personType VATR <totalTax>9.78</totalTax>

	should be rounded to 2 decimal places. The attribute is Mandatory if personType = VATR						Currently Mauritius MCD restaurants operates in only B2C mode so there is no VAT receipts or VAT invoices so this will be empty.

totalPrice	This attribute allows the seller to specify the total amount to be paid by the buyer. totalPrice should be numeric and it should be rounded to 2 decimal places.	totalPrice	String	20	19000	Mandatory	`<View taxMode="0"`
							`<ItemView>`
							`<totalPrice>75.00</totalPrice>`

Sample SALE JSON Payload:

```
{
    "buyerBRN": "",
    "buyerBusinessAddress": "",
    "buyerName": "",
    "buyerTAN": "",
    "buyerTypeMra": "",
    "cashierID": "Cashier ID",
    "currency": "MUR",
    "dateTimeInvoiceIssued": "20231009 11:08:27",
    "ebsCounterNo": "TERMINAL/COUNTER ID NO",
    "invoiceCounter": "INVOICE/CREDIT MEMO COUNTER ",
    "invoiceIdentifier": "Posted Document No",
    "invoiceRefIdentifier": "Applies to ID No",
    "invoiceTypeMra": " As per the MRA Specifications, see Data Structure ",
    "personTypeMra": " As per the MRA Specifications, see Data Structure ",
    "previousNoteHash": " HANDLED ON INVOX ",
    "reasonStated": "Reason Code for Credit Memo",
    "salesTransactions": "Payment Method Code",
    "sellerBRN": "Company Registration No",
    "sellerBusinessAddress": "Company Full Address",
    "sellerName": "Company Name",
    "sellerPhNo": "Company Phone Number",
    "sellerTAN": "Company TAN Number",
    "totalAmountPaid": "Total Amount Including VAT",
    "totalVatAmount": "Total VAT Amount",
    "totalAmtWoVatCur": "Total Amount Excluding VAT in MUR",
    "totalAmtWoVatMur": " Total Amount Excluding VAT in FCY ",
    "transactionTypeMra": "B2B",
    "noofitems": "Number of Item Lines",
    "invoiceTotal": " Total Amount Including VAT ",
    "mrainvoicelines": [
        {
            "amtWoVatCur": " Total Line Amount Excluding VAT in FCY ",
            "amtWoVatMur": " Total Line Amount Excluding VAT in MUR ",
            "discount": "Line Discount Percentage",
            "discountedValue": "Line Discount Amount",
            "itemDesc": "Line Item Description",
            "itemNo": "Item Line No",
            "natureMra": " As per the MRA Specifications, see Data Structure ",
            "productCodeMRA": " As per the MRA Specifications, see Data Structure ",
            "productCodeOwn": " As per the MRA Specifications, see Data Structure ",
            "quantity": "Line Quantity",
            "taxCodeMra": " As per the MRA Specifications, see Data Structure ",
            "totalPrice": "Total Line Amount",
            "unitPrice": "Unit Price",
            "vatAmount": "Total Line VAT Amount"
        },
        {
            "amtWoVatCur": "13500",
            "amtWoVatMur": "13500",
            "discount": "10",
            "discountedValue": "10800",
```

```
            "itemDesc": "Pedals",
            "itemNo": "2",
            "natureMra": "GOODS",
            "productCodeMRA": "m",
            "productCodeOwn": "m",
            "quantity": "10",
            "taxCodeMra": "TC01",
            "totalPrice": "13500",
            "unitPrice": "1200",
            "vatAmount": "3700"
        }
    ]
}
```

Error Codes and Information:

List of errors:

Code	Reason	HTTP Status Code	Description/Action
ERR0001	Authentication Error	400	Unauthorized request
ERR0020	Authentication Error	400	Missing header parameters **Action requested:** Please submit request with correct header parameters.
ERR0021	Invalid request header/body format	400	Missing body parameters **Action requested:** Please submit request with correct header parameters.
ERR0022	Invalid request header/body format	400	Below reasons could entail this error code • Request DateTime should not be blank • Request Id should not be blank • Encrypted invoice data should not be blank • Request Id should not exceed 50 characters • Request DateTime should not exceed 17 characters • token should not be blank • username should not be blank • ebsMraId should not be blank • token should not exceed 255 characters • username should not exceed 100 characters • ebsMraId should not exceed 50 characters **Action requested:** Please submit request with correct format
ERR0050	Authorisation Error	401	Token sent is not valid
ERR0100	EBS validation failed	400	Below reasons could entail this error code • EBS status is not active • EBS test status is **PASS_CONFIRMED**. EBS should be reset to be able to proceed with submission of invoice
ERR0200	Decryption failed	400	Could not decrypt **encryptedInvoice**
ERR0300	Failed Signature	400	Could not validate signature
ERR0400	Invoice data mapping error	400	Invalid JSON format for invoice data Note the maximum number of items should not exceed 2000 **Action requested:** Please build the invoice data as per the correct schema and resubmit the request.
ERR0500	Invalid TAN	200	TAN of registered user does not match with TAN of seller in invoice details. **Action requested:** Please ensure that the correct TAN has been set in the request JSON and resubmit the request.
ERR0600	Invoice Data Validation Error	200	The decrypted invoice data string could not be validated as per the e-Invoice JSON Schema provided by MRA. **Action requested:** Please build the invoice data as per the correct schema and resubmit the request. **"Please refer to The Data Structure of an e-Invoice in the Guidelines Section for the JSON schema"**
ERR0023	Internal Server Error	500	An internal server has occurred. **Action requested:** Please resubmit the request